

Adding a **percRecip** Effect to the Three-Way SAOM

Implementation Log — April 30, 2026

Jinwoo Cho

1 Overview

This note records every change made to the RSiena fork to add the *perceived reciprocity* effect (**percRecip**) and to set up a parameter-recovery study for it. Two layers of work were involved:

1. A new C++ effect class plus its registration in the build wiring. This is required because the statistic $\sum_i \sum_j x_{ij} x_{ji}$ is not expressible as a **crprod** of two existing dependent networks — the two “diagonals” it pairs come from the same simulated three-way object, so the existing effect framework cannot evaluate the contribution function without code-level support.
2. An R-side recovery harness driven by `siena07(simOnly = TRUE)` that exercises the new effect end-to-end against a known truth. Several iterations were needed to align the harness with the fork’s three-way mode.

Statistic. For perceiver i , sender j , receiver k , define the per-slice statistic

$$s_i = \sum_k x_{i,i,k} x_{i,k,i},$$

i.e., the count of alters m such that perceiver i believes there is a mutual tie between i and m . Aggregated across slices,

$$S = \sum_i s_i = \sum_i \sum_k x_{iik} x_{iki},$$

which is the perceived reciprocity statistic.

2 C++ Effect Class

Two new source files implement the effect, modeled on the existing **ThreeWayPerceiverEffect** pair:

- `src/model/effects/ThreeWayPerceivedReciprocityEffect.h`
- `src/model/effects/ThreeWayPerceivedReciprocityEffect.cpp`

The class derives directly from **NetworkEffect** (no covariate dependence), parses the perceiver index i from the variable name `Y[n]` at `initialize()` time, and defines the two required virtual methods.

Contribution function. With $\text{ego} = j$, $\text{alter} = k$, and perceiver i stored as `lperceiverIndex`,

$$\Delta s_i = \begin{cases} P_{k,i}^{(i)} & \text{if } j = i \quad (\text{toggling } i \rightarrow k) \\ P_{i,j}^{(i)} & \text{if } k = i \quad (\text{toggling } j \rightarrow i) \\ 0 & \text{otherwise.} \end{cases}$$

The corresponding code:

```

1 double ThreeWayPerceivedReciprocityEffect::calculateContribution(
2   int alter) const
3 {
4   const int ego = this->ego();
5   const int i   = this->lperceiverIndex;
6
7   if (ego == i) {
8     return this->inTieExists(alter) ? 1.0 : 0.0;
9   }
10  if (alter == i) {
11    return this->pNetwork()->tieValue(i, ego) != 0 ? 1.0 : 0.0;
12  }
13  return 0.0;
14 }
```

Evaluation statistic. `tieStatistic(alter)` sums to s_i when invoked over every existing tie of the slice. To avoid double-counting each reciprocated pair (visited once as (i, m) and once as (m, i)), the routine returns 1 only on the leg where `ego = lperceiverIndex`:

```

1 double ThreeWayPerceivedReciprocityEffect::tieStatistic(int alter)
2 {
3   const int ego = this->ego();
4   const int i   = this->lperceiverIndex;
5   if (ego == i && this->inTieExists(alter))
6     return 1.0;
7   return 0.0;
8 }
```

3 Build Wiring

The new files must be visible to the compiler, the effect factory, the package's master include, and the metadata table that drives the R side. Five files were touched.

1. src/model/effects/AllEffects.h. Added `#include "ThreeWayPerceivedReciprocityEffect.h"` at the bottom of the three-way SAOM block, alongside the existing perceiver-attribute effects.

2. src/model/effects/EffectFactory.cpp. A new branch in the `createEffect()` dispatch maps the short name to the C++ class:

```

1 else if (effectName == "percRecip")
2 {
3   pEffect = new ThreeWayPerceivedReciprocityEffect(pEffectInfo);
4 }
```

Inserted directly after the existing `percSenderAgree` branch.

3. Source-list files. The package uses a configure-time substitution from `src/sources.list` into `src/Makevars.in`. We appended `model/effects/ThreeWayPerceivedReciprocityEffect.cpp` to all three:

- `src/sources.list`
- `src/Makevars`
- `src/Makevars.win`

The first is the canonical source; the second is regenerated by `configure`; the third is the Windows build file.

4. data/allEffects.csv. A new metadata row registers the effect for the R side:

```
1 nonSymmetricObjective,perceived reciprocity,Number of perceiver-involved reciprocated ties,
   percRecip,TRUE,,,eval,FALSE,FALSE,FALSE,FALSE,FALSE,"",0,0,objective,NA,NA,0,0,0,0,dyadic,TRUE
```

Inserted directly after the standard `recip` row. The companion `data/allEffects.R` reads the CSV unchanged, so no additional edits there.

4 R Recovery Harness

The recovery script `recovery_percRecip.R` (project root) follows the standard two-stage pattern:

1. One `siena07(simOnly = TRUE, n3 = 50, returnDeps = TRUE)` call draws 50 wave-2 realizations under known truth.
2. A 50-iteration estimation loop fits each replicate with a fresh effects object and stores $\hat{\theta}$ together with its SE.

4.1 Final pilot configuration

- $n = 10$ actors, hence $K = 10$ perceiver slices.
- Two waves, 50 replications.
- `shareParameters = TRUE` on the threeway dependent variable: density, reciprocity, and `percRecip` share a single β across slices, materialized at `Y[shared]`.
- `Y[self]`: density and reciprocity (no cross-network `crprod` effects in the pilot).
- Rate parameters set explicitly (see Section ??).

Truth values:

$$\rho_{\text{perc}} = 6, \quad \rho_{\text{self}} = 6, \quad \beta_{\text{dens}}^{\text{perc}} = -2, \quad \beta_{\text{recip}}^{\text{perc}} = 1.5, \quad \beta_{\text{percRecip}} = 0.8, \quad \beta_{\text{dens}}^{\text{self}} = -2, \quad \beta_{\text{recip}}^{\text{self}} = 1.5.$$

4.2 Iterations during debugging

Several issues surfaced while the script was being aligned with the fork's three-way API. They are recorded here because each one masked the next.

(a) **Misuse of `sienaDependent(name = ...)`.** Initial drafts called `sienaDependent(arr, name = "Y[1]")`, which errored with “unused argument”. `sienaDependent` has no `name` parameter; `sienaDataCreate` reads variable names from its own call expression via `substitute()`. An interim fix used a `do.call` with a named list to bind the non-syntactic name “Y[1]”. This was later replaced (see (b)) by the fork’s native three-way constructor.

(b) **Wrong data construction for three-way mode.** The fork’s `sienaDependent` accepts a 4D array of shape (K, n, n, T) when `type = "threeway"` is supplied; the slice names and the self-net are derived automatically. The harness was rewritten to use this API:

```
1 Y <- sienaDependent(arr_4d, type = "threeway",
2                       shareParameters = TRUE)
3 dat <- sienaDataCreate(Y)
4 eff <- getEffects(dat)
```

(c) **Canonical row name under `shareParameters`.** `R/effects.r` (line 1007) renames the first occurrence of every shared structural row from `Y[1]` to `Y[shared]`. Calls of the form `setEffect(eff, density, na` therefore failed with “Effect not found”. The fix replaces `Y[1]` with `Y[shared]` for all canonical-slice `setEffect` and `includeEffects` calls.

(d) **Spurious `fix = TRUE` for simulation.** `fix` controls estimation; under `simOnly = TRUE` no estimation occurs and `fix` is irrelevant. All `fix = TRUE` flags were removed; only `initialValue` is needed to inject the true parameter values into the simulator.

(e) **Rate/density collinearity on `Y[self]`.** The first complete run produced

**** Standard errors not reliable ****

The following is approximately a linear combination for which the data carries no information:

$1\beta_{31} + 1\beta_{32}$.

The diagnostic `inc[c(31,32), c("name","shortName")]` identified the pair as `Y[self]` rate and `Y[self]` density. The root cause was that no rate parameters had been set in `truth`; the simulator used a default rate too small to produce identifiable movement on the self-net. Two new entries (`rate_perc`, `rate_self`) were added to `truth` and fed in via a per-slice loop:

```
1 for (k in seq_len(n_actors)) {
2   sim_eff <- setEffect(sim_eff, Rate, type = "rate",
3                       name = paste0("Y[", k, "]"),
4                       period = 1,
5                       initialValue = truth["rate_perc"])
6 }
7 sim_eff <- setEffect(sim_eff, Rate, type = "rate",
8                       name = "Y[self]", period = 1,
9                       initialValue = truth["rate_self"])
```

The loop is necessary because `shareParameters` only share-marks *structural* rows (`interaction1 == ""`); rate rows remain per-slice.

(f) **Silent name mismatch in the estimation loop.** `build_rep_data` originally bound the per-replicate dependent variable to the symbol `Yrep`, so the resulting slice names became `Yrep[shared]`, `Yrep[self]`, etc. The loop’s `includeEffects(..., name = "Y[shared]")` therefore matched no

row. `includeEffects` does not stop on a missing name — it emits a `cat` message and returns the table unchanged (R/`sienaeffects.r` lines 142–153). Across 50 replications all `percRecip` and `recip` additions silently no-opped, yielding 50 `percRecip-hat = NA` estimates. The fix is one line: bind the variable to `Y` inside `build_rep_data`, matching the base data.

5 Files Modified or Created

File	Status	Purpose
<code>src/model/effects/ThreeWayPerceivedReciprocityEffect.h</code>	new	class declaration
<code>src/model/effects/ThreeWayPerceivedReciprocityEffect.cpp</code>	new	class implementation
<code>src/model/effects/AllEffects.h</code>	edit	include the new header
<code>src/model/effects/EffectFactory.cpp</code>	edit	dispatch <code>percRecip</code> short name
<code>src/sources.list</code>	edit	add new <code>.cpp</code> to canonical list
<code>src/Makevars</code>	edit	regenerated from <code>Makevars.in</code> ; up
<code>src/Makevars.win</code>	edit	Windows source list
<code>data/allEffects.csv</code>	edit	metadata row for <code>percRecip</code>
<code>recovery_percRecip.R</code>	new	two-stage recovery harness
<code>notes/percRecip_changes.tex</code>	new	this note

6 Status

- C++ effect compiles cleanly under syntax-only check (`g++ -fsyntax-only`) against the package include path.
- Full installation via `devtools::load_all()` succeeds on the user’s machine.
- Simulation runs and produces valid wave-2 networks across all 50 replications.
- Estimation loop now correctly includes `percRecip` and the explicit rate parameters; the rate/-density collinearity warning has been addressed by setting non-default rates in `truth`.
- Pending: a clean recovery summary table showing recovered $\hat{\beta}_{\text{percRecip}} \approx 0.8$ with reasonable coverage. After confirmation, the pilot lifts to a multi-slice run with cross-network `crprod` effects re-enabled.